

## 1 Introdução

Este trabalho implementa e avalia perceptrons simples (Rosenblatt) para classificar duas letras, X e T, representadas como imagens binárias em uma grade  $5 \times 5$ . Cada pixel aceso vale  $+1$  e cada pixel apagado vale  $-1$  (codificação bipolar), de modo que cada letra é um vetor de entrada  $\mathbf{x} \in \{-1, +1\}^{25}$ . São resolvidos dois itens: (a) um único neurônio que decide entre as duas classes por meio de uma única saída bipolar; e (b) uma camada com dois neurônios, cada um especializado em reconhecer uma das letras, com codificação de saída do tipo “um-contratodos”. O código-fonte completo é apresentado no Apêndice A e também entregue em anexo como `perceptron_letras.py`.

## 2 Fundamentação teórica

### 2.1 O neurônio de McCulloch–Pitts / Perceptron

Um perceptron simples calcula uma combinação linear das entradas e aplica uma função de ativação não linear:

$$\text{net} = \sum_{i=1}^n w_i x_i + b = \mathbf{w} \cdot \mathbf{x} + b, \quad y = f(\text{net}) \quad (1)$$

em que  $x_i$  são as entradas (pixels da imagem),  $w_i$  os pesos sinápticos,  $b$  o *bias* (ou limiar, com entrada fixa igual a 1) e  $f(\cdot)$  a função de ativação. Como as classes deste problema são codificadas como  $-1$  e  $+1$ , usa-se a função degrau bipolar:

$$f(\text{net}) = \begin{cases} +1, & \text{net} \geq 0 \\ -1, & \text{net} < 0 \end{cases} \quad (2)$$

Geometricamente, a equação  $\mathbf{w} \cdot \mathbf{x} + b = 0$  define um hiperplano no espaço de entrada de dimensão  $n = 25$ ; o perceptron consegue separar corretamente as duas classes se, e somente se, elas forem *linearmente separáveis*. Como neste trabalho há apenas dois padrões de treinamento distintos (um por classe), a separação linear é sempre possível.

### 2.2 Regra de aprendizado de Rosenblatt

Os pesos são ajustados iterativamente, padrão a padrão, pela regra do perceptron:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(t - y)\mathbf{x}, \quad b \leftarrow b + \eta(t - y) \quad (3)$$

em que  $t \in \{-1, +1\}$  é a saída desejada (alvo),  $y$  é a saída atual do neurônio e  $\eta > 0$  é a taxa de aprendizado. Quando  $y = t$  nada é alterado; quando há erro,  $(t - y) = \pm 2$  e os pesos são deslocados na direção que reduz o erro daquele padrão. O *Teorema de Convergência do Perceptron* garante que, se as classes forem linearmente separáveis, esse procedimento converge em um número finito de épocas para pesos que classificam corretamente todo o conjunto de treinamento.

### 2.3 Extensão para múltiplos neurônios (item b)

Para discriminar  $K$  classes é comum usar uma camada com  $K$  neurônios de saída, cada um treinado para responder  $+1$  à sua classe e  $-1$  às demais (codificação bipolar “um-contratodos”). Sem camada oculta, cada neurônio  $k$  possui seu próprio vetor de pesos  $\mathbf{w}_k$  e *bias*  $b_k$ , e pode ser treinado de forma totalmente independente dos demais pela mesma regra da Equação (3), usando apenas a coluna de alvos correspondente à sua saída. É exatamente essa a estrutura pedida no item (b): dois neurônios, com saída desejada  $(y_1, y_2) = (+1, -1)$  para a letra X e  $(y_1, y_2) = (-1, +1)$  para a letra T.

### 3 Metodologia e implementação

#### 3.1 Padrões de entrada

As letras foram extraídas do enunciado como grades  $5 \times 5$  e convertidas em vetores de 25 posições (varredura linha a linha):

$$\mathbf{X} = \begin{bmatrix} 1 & -1 & -1 & -1 & 1 \\ -1 & 1 & -1 & 1 & -1 \\ -1 & -1 & 1 & -1 & -1 \\ -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & -1 & 1 \end{bmatrix} \quad \mathbf{T} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ -1 & -1 & 1 & -1 & -1 \\ -1 & -1 & 1 & -1 & -1 \\ -1 & -1 & 1 & -1 & -1 \\ -1 & -1 & 1 & -1 & -1 \end{bmatrix} \quad (4)$$

A representação visual desses padrões aparece na parte superior da Figura 1.

#### 3.2 Estrutura do código

O script `perceptron_letras.py` define a classe `Perceptron` (um neurônio, usada no item a) e a classe `CamadaPerceptrons` (composição de  $K$  perceptrons independentes, usada no item b). Ambas implementam o método `fit`, que repete a Equação (3) até que nenhum padrão do conjunto de treino seja classificado incorretamente em uma época completa (ou até um número máximo de épocas). O trecho abaixo mostra o núcleo do algoritmo (o código completo está no Apêndice A):

```
1 def fit(self, X, t, max_epocas=100):
2     for epoca in range(max_epocas):
3         erros = 0
4         for xi, ti in zip(X, t):
5             yi = self.predict(xi.reshape(1, -1))[0]
6             if yi != ti:
7                 erro = (ti - yi)
8                 self.w += self.eta * erro * xi
9                 self.b += self.eta * erro
10                erros += 1
11        self.historico_erros.append(erros)
12        if erros == 0:
13            break
```

Os pesos e o *bias* são inicializados em zero (inicialização clássica de Rosenblatt),  $\eta = 1$ , e a ativação bipolar da Equação (2) é aplicada em `predict`. Após o treinamento, o script ainda avalia a robustez dos modelos a ruído, invertendo aleatoriamente de 1 a 8 pixels dos padrões originais e medindo a taxa de acerto ao longo de 300 ensaios por nível de ruído. Cada execução do script gera uma única imagem consolidada (`saida/resultados.png`, Figura 1) com todos os resultados a seguir.

## 4 Resultados

#### 4.1 Convergência e pesos aprendidos

Ambos os modelos convergem em poucas épocas: o perceptron do item (a) em 3 épocas, e os dois neurônios do item (b) em 2–3 épocas cada (Figura 1, segunda linha). A Tabela 1 confirma que, ao final do treinamento, as saídas coincidem exatamente com os alvos da tabela do enunciado.

| Padrão | $y$ (item a) | alvo | $(y_1, y_2)$ (item b) | alvo     |
|--------|--------------|------|-----------------------|----------|
| X      | +1           | +1   | (+1, -1)              | (+1, -1) |
| T      | -1           | -1   | (-1, +1)              | (-1, +1) |

Table 1: Saídas obtidas após o treinamento vs. alvos desejados: 100% de acerto no conjunto de treino.

A terceira linha da Figura 1 mostra os pesos finais reorganizados na grade  $5 \times 5$  original. Nota-se que apenas os 12 pixels que *diferenciam* X de T recebem pesos não nulos ( $\pm 4$ , com  $\eta = 1$ ); os outros

13 pixels, idênticos nas duas letras (entre eles toda a linha central, que vale  $[-1, -1, +1, -1, -1]$  em ambas), não contribuem para a decisão e permanecem com peso zero – exatamente o comportamento esperado de um classificador linear treinado com apenas dois exemplos. Além disso, como o neurônio 1 do item (b) tem o mesmo alvo do neurônio do item (a), seus pesos são idênticos; e como o alvo do neurônio 2 é exatamente o oposto,  $\mathbf{w}_2 = -\mathbf{w}_1$ .

## 4.2 Robustez a ruído

Para avaliar a generalização, cada letra foi corrompida invertendo de 1 a 8 pixels aleatórios e reclassificada pelos modelos treinados (Figura 1, última linha). Ambos toleram bem o ruído leve (acerto de 100% até 5 pixels invertidos, de 25), degradando-se suavemente a partir daí. Esse limite é explicado analiticamente pela margem do hiperplano: com  $\eta = 1$  os 12 pesos não nulos valem  $\pm 4$ , de modo que  $\text{net} = \pm 48$  para os padrões originais, e cada pixel *discriminante* invertido altera  $\text{net}$  em 8 unidades. São necessários, portanto, ao menos 6 pixels discriminantes invertidos para cruzar a fronteira de decisão – e como os outros 13 pixels são irrelevantes para a decisão, inversões que caem sobre eles não afetam a classificação.

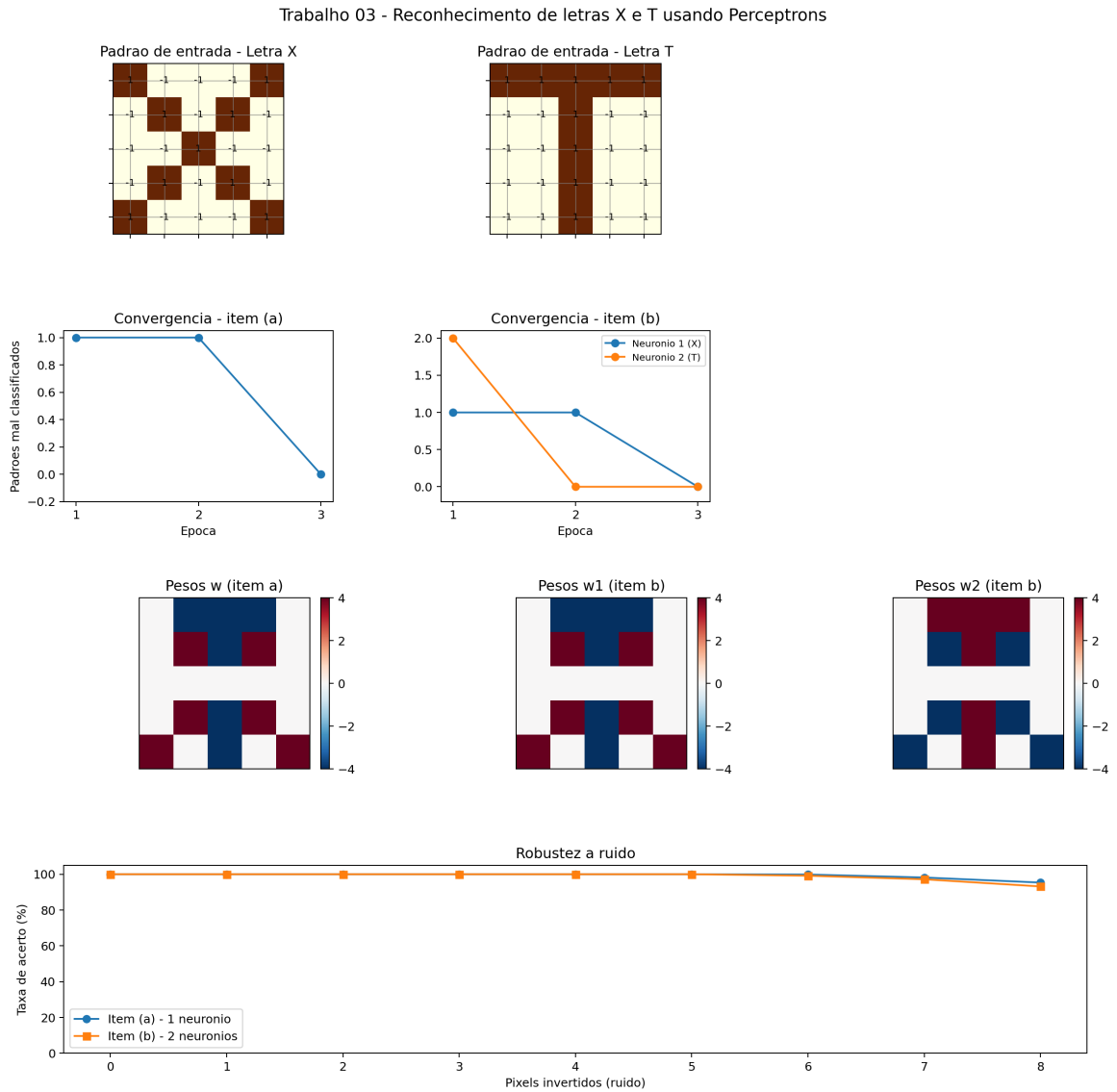


Figure 1: Painel único gerado pelo script a cada execução: padrões de entrada, convergência do treinamento, pesos finais reorganizados em grade  $5 \times 5$  e robustez a ruído (300 ensaios por nível de ruído).

## 5 Conclusão

Os dois perceptrons implementados resolvem o problema de reconhecer as letras X e T, pois o conjunto de treinamento com apenas dois padrões é trivialmente linearmente separável, satisfazendo as condições do Teorema de Convergência do Perceptron. O item (b), com dois neurônios independentes, é equivalente em capacidade ao item (a) para este problema binário – a única diferença é a codificação da saída (um único bit bipolar vs. dois bits redundantes) – mas generaliza melhor a estrutura para problemas com mais de duas classes, onde um único neurônio bipolar não seria suficiente. A análise dos pesos aprendidos mostrou que o perceptron identifica automaticamente, apenas pela regra de correção de erro, os pixels realmente discriminantes entre as duas letras, e o teste de ruído confirmou que essa solução é robusta a pequenas distorções nos padrões de entrada.

## A Código-fonte completo (perceptron\_letras.py)

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Trabalho 03 - Reconhecimento de letras (X e T) usando Perceptrons
5
6  Este script implementa:
7  (a) um Perceptron de 1 neurônio que discrimina entre as letras X e T;
8  (b) uma camada de Perceptrons com 2 neurônios (saída "one-hot" bipolar)
9  que faz a mesma discriminacao, um neurônio "especialista" em cada letra.
10
11 Os padroes de entrada sao grades 5x5 com pixels bipolares (+1 = aceso,
12 -1 = apagado), extraídos do enunciado (slide "Trabalho 03") e convertidos
13 em vetores de 25 posicoes (x1..x25), na ordem linha a linha (row-major).
14
15 Autor: gerado com apoio do Claude (Anthropic) para fins didaticos.
16 """
17
18 import os
19 import numpy as np
20 import matplotlib
21 matplotlib.use("Agg")
22 import matplotlib.pyplot as plt
23
24 # Reprodutibilidade dos experimentos com ruído
25 RNG = np.random.default_rng(42)
26
27 # Diretorio de saída para figuras e resultados
28 OUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "saida")
29 os.makedirs(OUT_DIR, exist_ok=True)
30
31
32 # -----
33 # 1. Definicao dos padroes (letras X e T em uma grade 5x5, valores bipolares)
34 # -----
35
36 X_GRID = np.array([
37     [ 1, -1, -1, -1, 1],
38     [-1, 1, -1, 1, -1],
39     [-1, -1, 1, -1, -1],
40     [-1, 1, -1, 1, -1],
41     [ 1, -1, -1, -1, 1],
42 ], dtype=float)
43
44 T_GRID = np.array([
45     [ 1, 1, 1, 1, 1],
46     [-1, -1, 1, -1, -1],
47     [-1, -1, 1, -1, -1],
48     [-1, -1, 1, -1, -1],
49     [-1, -1, 1, -1, -1],
50 ], dtype=float)
51
52 N_LIN, N_COL = X_GRID.shape
53 N_ENTRADAS = N_LIN * N_COL # 25
54
55 # Vetores de entrada (row-major flatten)
56 x_X = X_GRID.flatten()
```

```

57 x_T = T_GRID.flatten()
58
59 PADROES = np.array([x_X, x_T]) # shape (2, 25)
60 NOMES_PADROES = ["X", "T"]
61
62 # Alvos do item (a): 1 neurônio -> y = 1 se X, y = -1 se T
63 ALVOS_A = np.array([1.0, -1.0])
64
65 # Alvos do item (b): 2 neurônios -> (y1, y2) = (1,-1) se X, (-1,1) se T
66 ALVOS_B = np.array([
67     [1.0, -1.0], # saída desejada para o padrão X: (y1=1, y2=-1)
68     [-1.0, 1.0], # saída desejada para o padrão T: (y1=-1, y2=1)
69 ])
70
71
72 # -----
73 # 2. Função de ativação e Perceptron (regra de aprendizado de Rosenblatt)
74 # -----
75
76 def ativacao_bipolar(net):
77     """Função degrau bipolar: +1 se net >= 0, -1 caso contrário."""
78     return np.where(net >= 0, 1.0, -1.0)
79
80
81 class Perceptron:
82     """Perceptron simples (1 neurônio) com ativação bipolar (+1/-1).
83
84     Implementa a regra de aprendizado de Rosenblatt:
85         w <- w + eta * (t - y) * x
86         b <- b + eta * (t - y)
87     onde t é o alvo, y = f(w.x + b) é a saída atual e eta é a taxa de
88     aprendizado.
89     """
90
91     def __init__(self, n_entradas, eta=1.0):
92         # pesos e bias iniciais em zero (inicialização clássica do
93         # perceptron de Rosenblatt). A regra de aprendizado converge a
94         # partir de qualquer ponto inicial quando o problema é linearmente
95         # separável, mas começar do zero torna o processo determinista e
96         # deixa claro, no vetor de pesos final, exatamente quais entradas
97         # foram ajustadas para separar as duas classes.
98         self.w = np.zeros(n_entradas)
99         self.b = 0.0
100         self.eta = eta
101         self.historico_erros = [] # número de padrões mal-classificados por época
102
103     def net(self, X):
104         return X @ self.w + self.b
105
106     def predict(self, X):
107         return ativacao_bipolar(self.net(X))
108
109     def fit(self, X, t, max_epocas=100):
110         """Treina o perceptron até convergência (erro zero) ou max_epocas."""
111         for epoca in range(max_epocas):
112             erros = 0
113             for xi, ti in zip(X, t):
114                 yi = self.predict(xi.reshape(1, -1))[0]
115                 if yi != ti:
116                     erro = (ti - yi)
117                     self.w += self.eta * erro * xi
118                     self.b += self.eta * erro
119                     erros += 1
120             self.historico_erros.append(erros)
121             if erros == 0:
122                 break
123         return self
124
125
126 class CamadaPerceptrons:
127     """Camada de M perceptrons independentes (sem camada oculta),
128     usada no item (b) com M = 2 neurônios de saída. Cada neurônio da
129     camada é treinado de forma independente com a regra do perceptron,
130     usando a mesma entrada e sua própria coluna de alvo.
131     """

```

```

132
133     def __init__(self, n_entradas, n_saidas, eta=1.0):
134         self.neuronios = [Perceptron(n_entradas, eta=eta)
135                             for _ in range(n_saidas)]
136
137     def predict(self, X):
138         saidas = [n.predict(X) for n in self.neuronios]
139         return np.stack(saidas, axis=-1)
140
141     def fit(self, X, T, max_epocas=100):
142         """T tem shape (n_amostras, n_saidas)."""
143         for k, neuronio in enumerate(self.neuronios):
144             neuronio.fit(X, T[:, k], max_epocas=max_epocas)
145         return self
146
147
148 # -----
149 # 3. Treinamento
150 # -----
151
152 def treinar_item_a():
153     p = Perceptron(N_ENTRADAS, eta=1.0)
154     p.fit(PADROES, ALVOS_A, max_epocas=100)
155     return p
156
157
158 def treinar_item_b():
159     camada = CamadaPerceptrons(N_ENTRADAS, n_saidas=2, eta=1.0)
160     camada.fit(PADROES, ALVOS_B, max_epocas=100)
161     return camada
162
163
164 # -----
165 # 4. Teste de robustez a ruído (pixels invertidos aleatoriamente)
166 # -----
167
168 def aplicar_ruído(x, n_flips, rng):
169     """Retorna uma cópia de x com n_flips posições com o sinal invertido."""
170     x_ruidoso = x.copy()
171     idx = rng.choice(len(x), size=n_flips, replace=False)
172     x_ruidoso[idx] *= -1
173     return x_ruidoso
174
175
176 def testar_robustez_a(perceptron, n_ensaios=300, max_flips=8):
177     """Para cada quantidade de pixels invertidos (0..max_flips), gera
178     n_ensaios versões ruidosas de X e de T e mede a taxa de acerto do
179     perceptron do item (a)."""
180     taxas = []
181     for n_flips in range(0, max_flips + 1):
182         acertos = 0
183         total = 0
184         for x_original, alvo in zip(PADROES, ALVOS_A):
185             for _ in range(n_ensaios):
186                 x_ruido = aplicar_ruído(x_original, n_flips, RNG) if n_flips > 0 else x_original
187                 y = perceptron.predict(x_ruido.reshape(1, -1))[0]
188                 acertos += int(y == alvo)
189                 total += 1
190             taxas.append(acertos / total)
191     return np.array(taxas)
192
193
194 def testar_robustez_b(camada, n_ensaios=300, max_flips=8):
195     taxas = []
196     for n_flips in range(0, max_flips + 1):
197         acertos = 0
198         total = 0
199         for x_original, alvo in zip(PADROES, ALVOS_B):
200             for _ in range(n_ensaios):
201                 x_ruido = aplicar_ruído(x_original, n_flips, RNG) if n_flips > 0 else x_original
202                 y = camada.predict(x_ruido.reshape(1, -1))[0]
203                 acertos += int(np.array_equal(y, alvo))
204                 total += 1
205             taxas.append(acertos / total)
206     return np.array(taxas)

```

```

207
208
209 # -----
210 # 5. Grafico (uma unica imagem consolidada com todos os resultados)
211 # -----
212
213 CMAP_PADRAO = "YlOrBr"
214 CMAP_PESOS = "RdBu_r"
215
216
217 def gerar_figura_resultados(p, camada, taxas_a, taxas_b, nome_arquivo="resultados.png"):
218     """Monta e salva UMA UNICA imagem com todos os resultados do trabalho:
219     padroes de entrada, curvas de convergencia, pesos aprendidos e o teste
220     de robustez a ruido. E sobrescrita a cada execucao do script, entao o
221     arquivo de saida reflete sempre o resultado mais recente.
222     """
223     # historico de erros do item (b), com as duas listas niveladas no
224     # mesmo tamanho (preenchendo com 0 apos a convergencia de cada uma)
225     n_epocas_b = max(len(n.historico_erros) for n in camada.neuronios)
226     hist_b = np.array([
227         n.historico_erros + [0] * (n_epocas_b - len(n.historico_erros))
228         for n in camada.neuronios
229     ])
230
231     fig = plt.figure(figsize=(13, 13))
232     gs = fig.add_gridspec(4, 3, height_ratios=[1, 1, 1, 1.1], hspace=0.55, wspace=0.4,
233         top=0.93, bottom=0.04, left=0.06, right=0.98)
234
235     # ---- linha 0: padroes de entrada (X e T) ----
236     for col, (grid, nome) in enumerate(zip([X_GRID, T_GRID], NOMES_PADROES)):
237         ax = fig.add_subplot(gs[0, col])
238         ax.imshow(grid, cmap=CMAP_PADRAO, vmin=-1, vmax=1)
239         ax.set_title(f"Padrao de entrada - Letra {nome}")
240         ax.set_xticks(range(N_COL)); ax.set_yticks(range(N_LIN))
241         ax.set_xticklabels([]); ax.set_yticklabels([])
242         for i in range(N_LIN):
243             for j in range(N_COL):
244                 ax.text(j, i, int(grid[i, j]), ha="center", va="center", fontsize=8)
245         ax.grid(color="gray", linewidth=0.5)
246
247     # ---- linha 1: convergencia (item a e item b) ----
248     ax = fig.add_subplot(gs[1, 0])
249     h_a = p.historico_erros
250     ax.plot(range(1, len(h_a) + 1), h_a, marker="o", color="tab:blue")
251     ax.set_title("Convergencia - item (a)")
252     ax.set_xlabel("Epoca"); ax.set_ylabel("Padroes mal classificados")
253     ax.set_xticks(range(1, len(h_a) + 1))
254     ax.set_ylim(bottom=-0.2)
255
256     ax = fig.add_subplot(gs[1, 1])
257     for h, r in zip(hist_b, ["Neuronio 1 (X)", "Neuronio 2 (T)"]):
258         ax.plot(range(1, len(h) + 1), h, marker="o", label=r)
259     ax.set_title("Convergencia - item (b)")
260     ax.set_xlabel("Epoca")
261     ax.set_xticks(range(1, n_epocas_b + 1))
262     ax.set_ylim(bottom=-0.2)
263     ax.legend(fontsize=8)
264
265     # ---- linha 2: pesos aprendidos (item a, e os 2 neuronios do item b) ----
266     pesos = [p.w, camada.neuronios[0].w, camada.neuronios[1].w]
267     titulos_pesos = ["Pesos w (item a)", "Pesos w1 (item b)", "Pesos w2 (item b)"]
268     vlim = max((np.max(np.abs(w)) for w in pesos), default=1) or 1
269     for col, (w, titulo) in enumerate(zip(pesos, titulos_pesos)):
270         ax = fig.add_subplot(gs[2, col])
271         im = ax.imshow(w.reshape(N_LIN, N_COL), cmap=CMAP_PESOS, vmin=-vlim, vmax=vlim)
272         ax.set_title(titulo)
273         ax.set_xticks([]); ax.set_yticks([])
274         fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
275
276     # ---- linha 3: robustez a ruido (ocupando as 3 colunas) ----
277     ax = fig.add_subplot(gs[3, :])
278     flips = range(len(taxas_a))
279     ax.plot(flips, taxas_a * 100, marker="o", label="Item (a) - 1 neuronio")
280     ax.plot(flips, taxas_b * 100, marker="s", label="Item (b) - 2 neuronios")
281     ax.set_xlabel("Pixels invertidos (ruido)")

```

```

282 ax.set_ylabel("Taxa de acerto (%)")
283 ax.set_title("Robustez a ruido")
284 ax.set_ylim(0, 105)
285 ax.legend()
286
287 fig.suptitle("Trabalho 03 - Reconhecimento de letras X e T usando Perceptrons",
288             fontsize=14, y=0.98)
289
290 caminho = os.path.join(OUT_DIR, nome_arquivo)
291 fig.savefig(caminho, dpi=180)
292 plt.close(fig)
293 return caminho
294
295
296 # -----
297 # 6. Execucao principal
298 # -----
299
300 def main():
301     linhas_log = []
302
303     def log(msg=""):
304         print(msg)
305         linhas_log.append(str(msg))
306
307     log("=" * 70)
308     log("Trabalho 03 - Reconhecimento de letras X e T usando Perceptrons")
309     log("=" * 70)
310
311     log("\nPadroes de entrada (vetores bipolares de 25 posicoes):")
312     log(f" x_X = {x_X.astype(int).tolist()}")
313     log(f" x_T = {x_T.astype(int).tolist()}")
314
315     # ----- Item (a): 1 neuronio -----
316     log("\n" + "-" * 70)
317     log("ITEM (a): Perceptron de 1 neuronio")
318     log("-" * 70)
319     p = treinar_item_a()
320     n_epocas_a = len(p.historico_erros)
321     log(f"Convergiu em {n_epocas_a} epoca(s). Historico de erros por epoca: {p.historico_erros}")
322     log(f"Pesos finais (w): {np.round(p.w, 3).tolist()}")
323     log(f"Bias final (b): {round(p.b, 3)}")
324
325     saida_X = p.predict(x_X.reshape(1, -1))[0]
326     saida_T = p.predict(x_T.reshape(1, -1))[0]
327     log(f"Saida para o padrao X: y = {saida_X:+.0f} (esperado +1)")
328     log(f"Saida para o padrao T: y = {saida_T:+.0f} (esperado -1)")
329
330     # ----- Item (b): 2 neuronios -----
331     log("\n" + "-" * 70)
332     log("ITEM (b): Camada com 2 neuronios (Perceptrons independentes)")
333     log("-" * 70)
334     camada = treinar_item_b()
335     for k, n in enumerate(camada.neuronios, start=1):
336         log(f"Neuronio {k}: convergiu em {len(n.historico_erros)} epoca(s); "
337             f"erros por epoca = {n.historico_erros}")
338         log(f" Pesos w{k}: {np.round(n.w, 3).tolist()}")
339         log(f" Bias b{k}: {round(n.b, 3)}")
340
341     saida_X_b = camada.predict(x_X.reshape(1, -1))[0]
342     saida_T_b = camada.predict(x_T.reshape(1, -1))[0]
343     log(f"Saida (y1,y2) para o padrao X: {tuple(saida_X_b)} (esperado (+1,-1))")
344     log(f"Saida (y1,y2) para o padrao T: {tuple(saida_T_b)} (esperado (-1,+1))")
345
346     # ----- Robustez a ruido -----
347     log("\n" + "-" * 70)
348     log("Teste de robustez a ruido (inversao aleatoria de pixels)")
349     log("-" * 70)
350     taxas_a = testar_robustez_a(p)
351     taxas_b = testar_robustez_b(camada)
352     for k in range(len(taxas_a)):
353         log(f" {k} pixel(s) invertido(s): acerto item (a) = {taxas_a[k]*100:5.1f}% | "
354             f"item (b) = {taxas_b[k]*100:5.1f}%")
355
356     # ----- Figura unica com todos os resultados -----

```



```

357     caminho_fig = gerar_figura_resultados(p, camada, texas_a, texas_b, "resultados.png")
358
359     # ----- Salvar log -----
360     with open(os.path.join(OUT_DIR, "resultados.txt"), "w", encoding="utf-8") as f:
361         f.write("\n".join(linhas_log))
362
363     log(f"\nImagem consolidada salva em: {caminho_fig}")
364     log(f"Log salvo em: {os.path.join(OUT_DIR, 'resultados.txt')}")
365
366
367 if __name__ == "__main__":
368     main()

```